Щучкин Максим, Б07-202

Отчет по лабораторной работе #1

«Измерение производительности компьютера на стандартных операциях»

Цель: на основе примера ex02b.c измерить производительность персонального компьютера и учебного сервера класса на операциях следующих типов: сложение, вычитание, умножение, деление, возведение в степень (paw), а также на вычислениях функций математической библиотеки – exp, log, sin.

Текст программы:

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    long nc = 1000000000;
    double t1, t2, s1 = 0.0, s2 = 0.0;
    double a = 1.23456789, b = 0.987654321;

    /* Защита от оптимизации: используем аргумент командной строки */
    volatile double sink = (argc > 1) ? atof(argv[1]) : 1.0;

    /* Деление */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += a / (b + i * sink);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += a / (i * sink);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec '/' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Умножение */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += a * (b + i * sink);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += a * (i * sink);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec '*' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Сложение */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += a + b + i * sink;
```

```
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += a + i * sink;
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec '+' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Вычитание */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += a - b - i * sink;
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += a - i * sink;
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec '-' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Возведение в степень */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += pow(a * sink, b);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += pow(a * sink, i * 1e-9);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec 'pow' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Экспонента */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += exp(b * sink);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += exp(i * 1e-9 * sink);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec 'exp' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Логарифм */
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += log(b * sink + 1.0);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += log(i * 1e-9 * sink + 1.0);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec 'log' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Синус */
```

```
    t1 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s1 += sin(b * sink);
    t1 = omp_get_wtime() - t1;
    t2 = omp_get_wtime();
    for(long i = 1; i <= nc; i++) s2 += sin(i * 1e-9 * sink);
    t2 = omp_get_wtime() - t2;
    printf("Time: %.6lf %.6lf sec 'sin' perf.: %e GFlops\n", t1, t2, 1.0 /
fabs(t2 - t1));

    /* Финальная защита: выводим результат, зависящий от всех вычислений */
    printf("Sink: %.12lf\n", s1 + s2);
    return 0;
}
```

Output:

```
fnbik09@sr210:~/mpc/task1$ mpicc -O2 -fopenmp solution_01.c -lm -o solution_01.px
solution_01.c: In function ‘main’:
solution_01.c:11:41: warning: implicit declaration of function ‘atof’ [-Wimplicit-function-declaration]
   11 |     volatile double sink = (argc > 1) ? atof(argv[1]) : 1.0;
      |                                         ^~~~
fnbik09@sr210:~/mpc/task1$ ./solution_01.px 1.0
Time: 3.156985 2.870406 sec '/' perf.: 3.489445e+00 GFlops
Time: 2.817990 2.794338 sec '*' perf.: 4.227934e+01 GFlops
Time: 2.802747 2.851806 sec '+' perf.: 2.038362e+01 GFlops
Time: 2.815104 2.813294 sec '-' perf.: 5.527492e+02 GFlops
Time: 8.240666 8.637941 sec 'pow' perf.: 2.517146e+00 GFlops
Time: 5.453392 5.995392 sec 'exp' perf.: 1.845017e+00 GFlops
Time: 4.549723 4.785800 sec 'log' perf.: 4.235904e+00 GFlops
Time: 4.168250 4.502247 sec 'sin' perf.: 2.994047e+00 GFlops
Sink: inf
```

Выводы:

1) Базовые операции (+, -, *, /) выполняются непосредственно блоками FPU процессора и достигают десятков/сотен GFlops. Трансцендентные функции (pow, exp, log, sin) реализованы в программной библиотеке libm через итерационные алгоритмы (ряды Тейлора, методы Ньютона, CORDIC), поэтому их производительность на 1–2 порядка ниже (1–5 GFlops).
2) Компиляторы с флагами -O2/-O3 агрессивно вырезают циклы. Без volatile, зависимости от argv или принудительного вывода результата бенчмарк покажет 0.000000 sec и бесконечную производительность, что делает измерение бессмысленным.
3) Почему минуч показал ~552 GFlops, а плюс ~20 GFlops? Возможно, компилятор векторизовал цикл вычитания, обрабатывая 4–8 double за такт? Не очень уверен.